Jenkins Mastery

Declarative Pipeline

Pipeline as Code · Groovy · Stages · Stage View · Console Output

---

Saleem Ahmed
linkedin.com/in/saleemfai · Cameroon
Jenkins 2.541.2 · Groovy DSL · April 2026

Table of Contents

1. Overview — What Is a Declarative Pipeline?
2. The Anatomy of a Declarative Pipeline
3. Part 1 — Create a Pipeline Job and Write a Hello World Script
4. Part 2 — Build a Multi-Stage Pipeline with Shell Commands
5. Part 3 — Run the Pipeline and Read the Stage View
6. Part 4 — Read the Console Output
7. Key Lessons Learned
8. Quick Reference — Declarative Pipeline Syntax

1. Overview — What Is a Declarative Pipeline?

A **Declarative Pipeline** in Jenkins is a structured way to define an entire CI/CD workflow as code — written in Groovy and stored directly in Jenkins or in a `Jenkinsfile` committed to your source repository. Unlike Freestyle jobs where you click through a UI to add build steps one at a time, a Declarative Pipeline describes the complete workflow — all its stages, steps, agents, and conditions — in a single, version-controlled script.

The Declarative syntax is Jenkins' recommended approach. It enforces a clean, readable structure, is validated by Jenkins at parse time before any command runs, and makes complex pipelines maintainable by anyone who can read code.

Feature	Freestyle Job	Declarative Pipeline
Configuration	UI click-through	Groovy code in a script editor or Jenkinsfile
Version control	Not possible (stored in Jenkins DB)	Yes — commit Jenkinsfile to Git
Stage visibility	No Stage View	Full Stage View with per-stage timing
Reusability	Copy-paste jobs	Shared libraries, parameterised scripts
Restart from stage	Not available	Yes — restart any failed stage independently

2. The Anatomy of a Declarative Pipeline

Every Declarative Pipeline follows the same top-level structure. Jenkins validates this structure at parse time — a missing required block causes the build to fail before a single command is executed.

```
pipeline {
    agent any          // Where to run — any available Jenkins agent
    stages {           // Container for all pipeline stages
        stage('Name') {  // A named stage — appears in Stage View
            steps {      // The actual commands to run
                echo 'Hello'
            }
        }
    }
}
```

Block	Required	Purpose
`pipeline { }`	■ Yes	Outer wrapper — every Declarative Pipeline must start with this
`agent any`	■ Yes	Tells Jenkins to run on any available executor
`stages { }`	■ Yes	Contains all stage() blocks — must have at least one
`stage('Name') { }`	■ Yes (≥1)	Named unit of work — the name appears as a column in Stage View
`steps { }`	■ Yes	Contains the actual commands inside each stage
`echo 'text'`	Optional	Jenkins built-in — prints text to Console Output, no shell invoked
`sh 'command'`	Optional	Runs a shell command on the Jenkins agent OS

3. Part 1 — Create a Pipeline Job and Write a Hello World Script

PART 1

Create a Pipeline Job and Write a Hello World Script

The first step is creating a Pipeline job — a different job type from Freestyle. Pipeline jobs have a dedicated **Pipeline** configuration section where the Groovy script is entered, and they produce the Stage View UI that Freestyle jobs do not have.

1 Create the Pipeline job

```
Jenkins Dashboard → New Item
→ Enter name: pipelines-code-example
→ Pipeline
→ OK
```

2 Navigate to the Pipeline section and set the definition

Inside **pipelines-code-example** → **Configure**, click **Pipeline** in the left sidebar. Set the **Definition** dropdown to:

```
Pipeline script
```

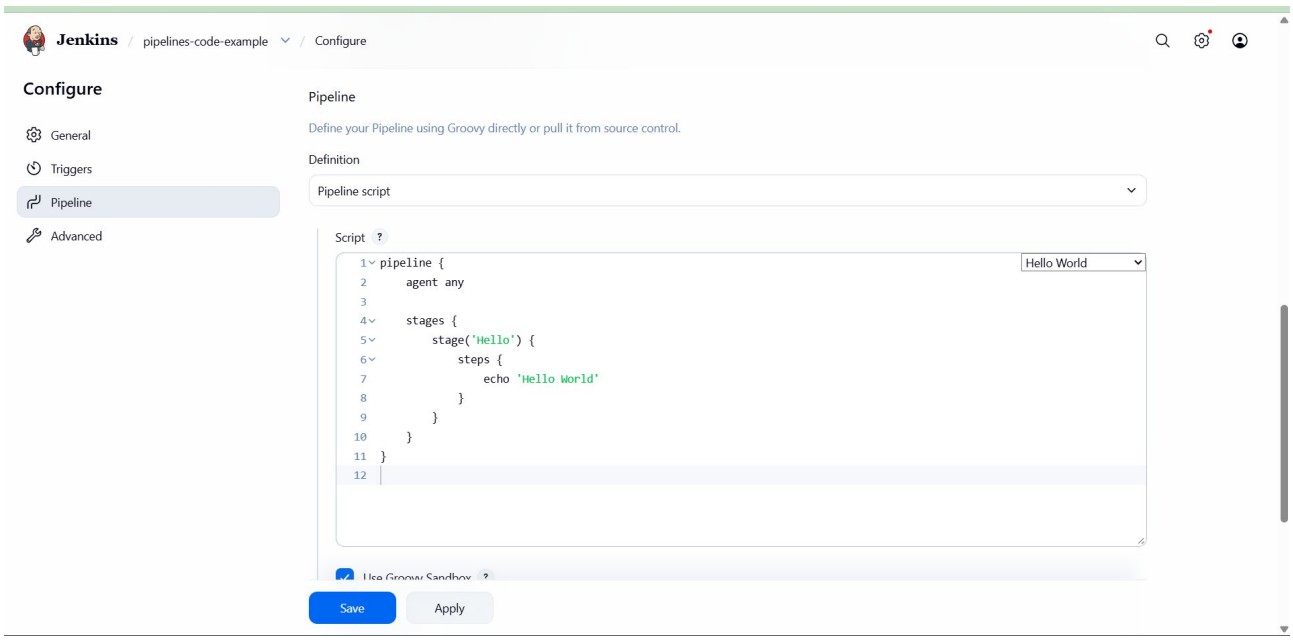
This tells Jenkins the pipeline will be written directly in the browser's script editor. The alternative — *Pipeline script from SCM* — pulls a `Jenkinsfile` from a Git repository.

3 Load the Hello World template and review the script

In the **Script** editor, click the **Hello World** dropdown in the top-right corner to load the minimal valid Declarative Pipeline template:

```
pipeline {
    agent any

    stages {
        stage('Hello') {
            steps {
                echo 'Hello World'
            }
        }
    }
}
```



pipelines-code-example Configure → Pipeline — Definition set to 'Pipeline script', Hello World template loaded in the Groovy script editor, Use Groovy Sandbox checked

Line	What it does
`pipeline { }`	Declares this as a Declarative Pipeline
`agent any`	Run on any available Jenkins executor or node
`stages { }`	Wrapper for all stages in the pipeline
`stage('Hello') { }`	A single stage named 'Hello' — visible in Stage View
`steps { }`	Contains the commands to run in this stage
`echo 'Hello World'`	Prints 'Hello World' to the Console Output

Clicked **Save** to apply the configuration.

Note: Use Groovy Sandbox (checked by default) restricts the script to a safe subset of Groovy. This prevents pipeline scripts from executing arbitrary system calls that could compromise the Jenkins server. For most pipelines, the sandbox is sufficient and recommended.

4. Part 2 — Build a Multi-Stage Pipeline with Shell Commands

PART 2

Build a Multi-Stage Pipeline with Shell Commands

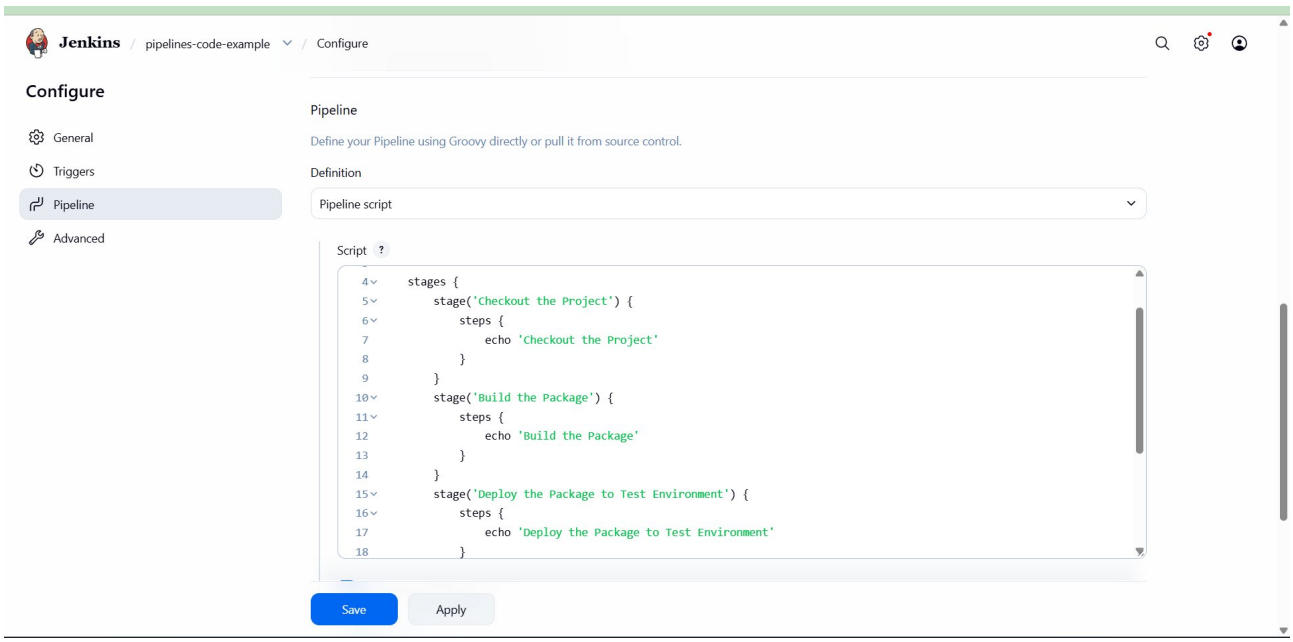
A single-stage Hello World pipeline demonstrates syntax but doesn't model a real CI/CD workflow. In practice, pipelines have multiple sequential stages — each representing a distinct phase of the delivery process. This part extends the pipeline to four stages and introduces `sh` steps to run real shell commands inside a stage.

4 Update the pipeline script to four stages

Inside **pipelines-code-example** → **Configure** → **Pipeline**, replaced the Hello World script with the following four-stage Declarative Pipeline:

```
pipeline {
    agent any

    stages {
        stage('Checkout the Project') {
            steps {
                sh 'date'
                sh 'pwd'
            }
        }
        stage('Build the Package') {
            steps {
                echo 'Build the Package'
            }
        }
        stage('Deploy the Package to Test Environment') {
            steps {
                echo 'Deploy the Package to Test Environment'
            }
        }
        stage('Deploy the Package to Prod Environment') {
            steps {
                echo 'Deploy the Package to Prod Environment'
            }
        }
    }
}
```



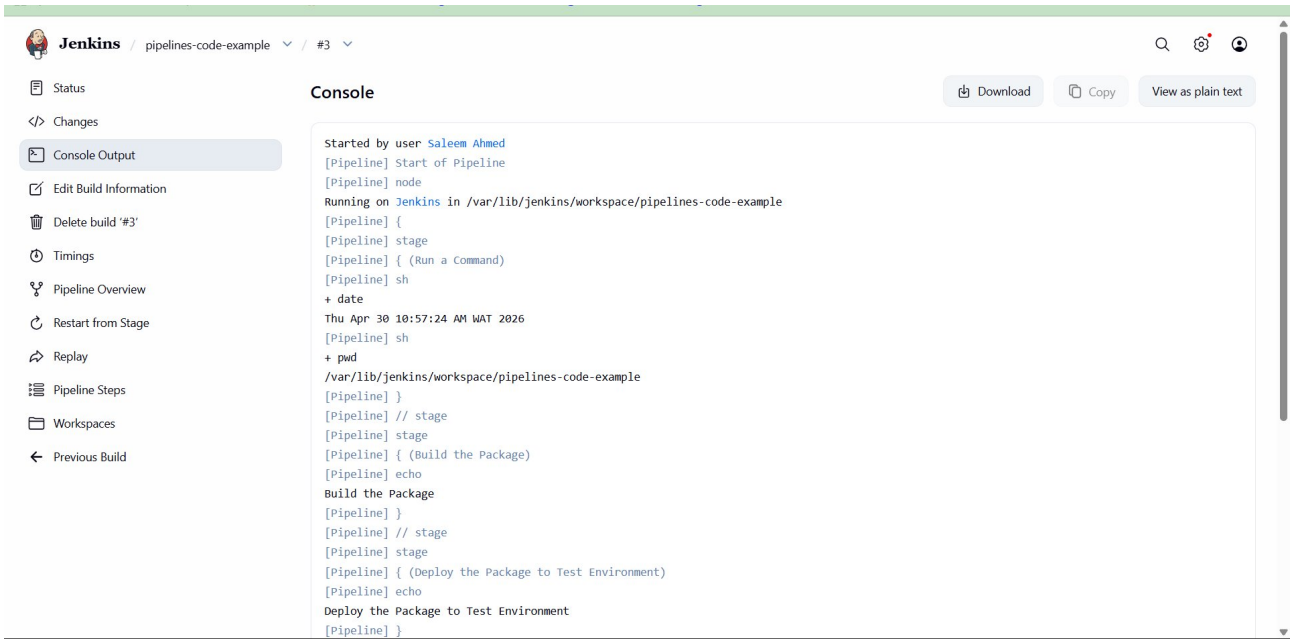
■ *pipelines-code-example Configure → Pipeline — four-stage Declarative Pipeline script showing stages: Checkout the Project (sh steps), Build the Package, Deploy to Test Environment (lines 4–18 visible in editor)*

Breaking Down the Four Stages

Stage	Steps Used	Models in CI/CD
Checkout the Project	`sh 'date'`, `sh 'pwd'`	Source code checkout — shell commands confirm workspace and timestamp
Build the Package	`echo`	Compilation and packaging phase (Maven/Gradle in real pipelines)
Deploy to Test Environment	`echo`	Deployment to test/staging environment
Deploy to Prod Environment	`echo`	Production deployment — final promotion gate

The sh Step vs echo

Step	What it does	Console Output marker
`sh 'command'`	Executes a shell command on the agent OS. stdout/stderr appear in log.	Prefixed with + (bash xtrace)
`echo 'text'`	Jenkins built-in — prints text to the log. No shell spawned.	No + prefix — plain text line



■ *pipelines-code-example Configure → Pipeline — same four-stage script in wider browser window, full Configure page layout visible with left sidebar navigation*

Clicked **Save** to apply.

> **Tip:** In a real Jenkins pipeline, stage('Checkout the Project') would use the checkout scm step (or git url: '...') to clone the repository. The sh 'date' and sh 'pwd' steps here serve as observable shell commands to demonstrate that sh runs real OS commands inside the workspace.

5. Part 3 — Run the Pipeline and Read the Stage View

PART 3

Run the Pipeline and Read the Stage View

With the four-stage pipeline saved, triggering a build produces a Stage View on the job's status page — a visual grid showing each stage as a column with colour-coded build status and individual execution timing per stage.

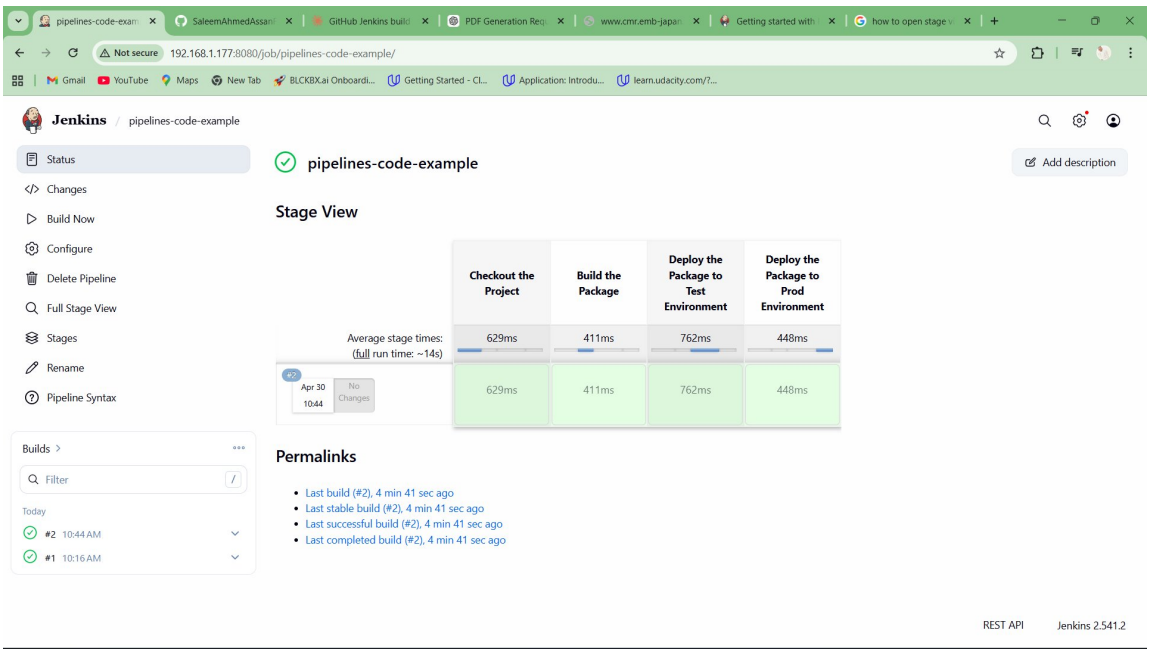
5 Trigger the pipeline

Clicked **Build Now** from the **pipelines-code-example** status page. Jenkins queued and executed the pipeline, running all four stages sequentially on the Jenkins built-in node.

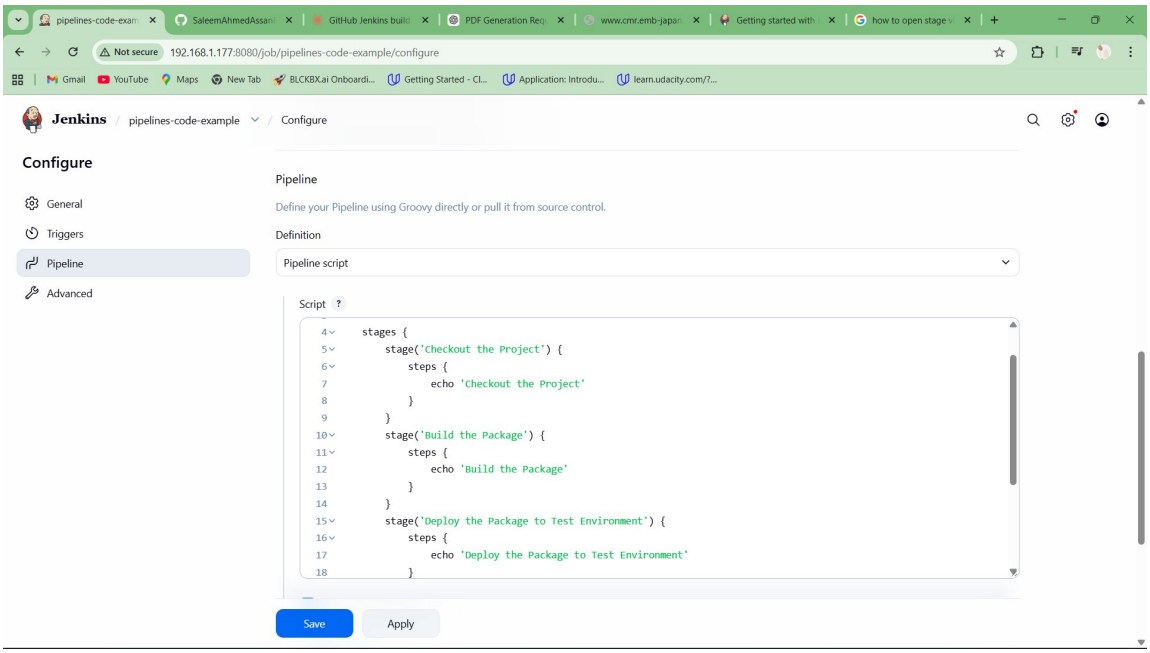
6 Read the Stage View after Build #2

After Build **#2** completed (Apr 30, 10:44), the Stage View showed all four stages as columns with green cells — indicating every stage passed:

Stage	Time (Build #2)	Result
Checkout the Project	629 ms	■ SUCCESS
Build the Package	411 ms	■ SUCCESS
Deploy the Package to Test Environment	762 ms	■ SUCCESS
Deploy the Package to Prod Environment	448 ms	■ SUCCESS
Full run time	**~14 seconds**	**■ Overall SUCCESS**



■ *pipelines-code-example Status — Stage View showing Build #2 (Apr 30 10:44) with all four stages in green, average stage times in the header row, Builds #1 and #2 both showing ■ in the Builds sidebar*



■ *Same Stage View in wider browser — full pipeline job page with left sidebar navigation (Status, Changes, Build Now, Configure, Pipeline Syntax), Stage View grid, and Permalinks section*

Reading the Stage View Grid

Element	Location	What it shows
Column headers	Top row	Stage names — exactly as written in stage('...')
Average times row	Below headers	Mean duration for that stage across all runs

Build row	Main grid area	Per-build, per-stage duration. Green = passed, Red = failed
Build label (left)	Left column	Build number, date, time, No Changes indicator
Builds sidebar	Left panel	All builds with ■/■ status icons and timestamps

Result: Both builds completed successfully. The Stage View provided an at-a-glance health dashboard — making it immediately clear which stage was slowest (Deploy to Test at 762ms) and whether any stage failed. The average row accumulates across builds, giving a performance baseline for detecting regressions in later runs.

6. Part 4 — Read the Console Output

PART 4

Read the Console Output

The Console Output provides a line-by-line execution trace of everything Jenkins and the pipeline script did during a build. It is the primary diagnostic tool when a build fails — every shell command run, every echo printed, and every Jenkins framework action is logged here.

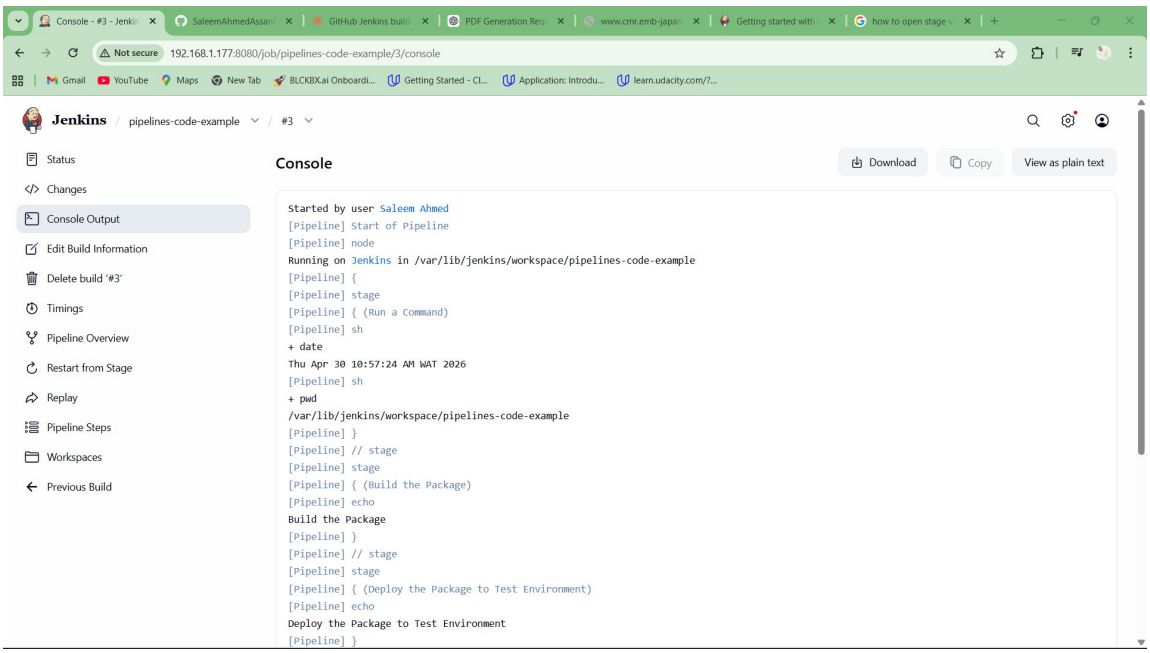
7 Open the Console Output for Build #3

Navigate to:

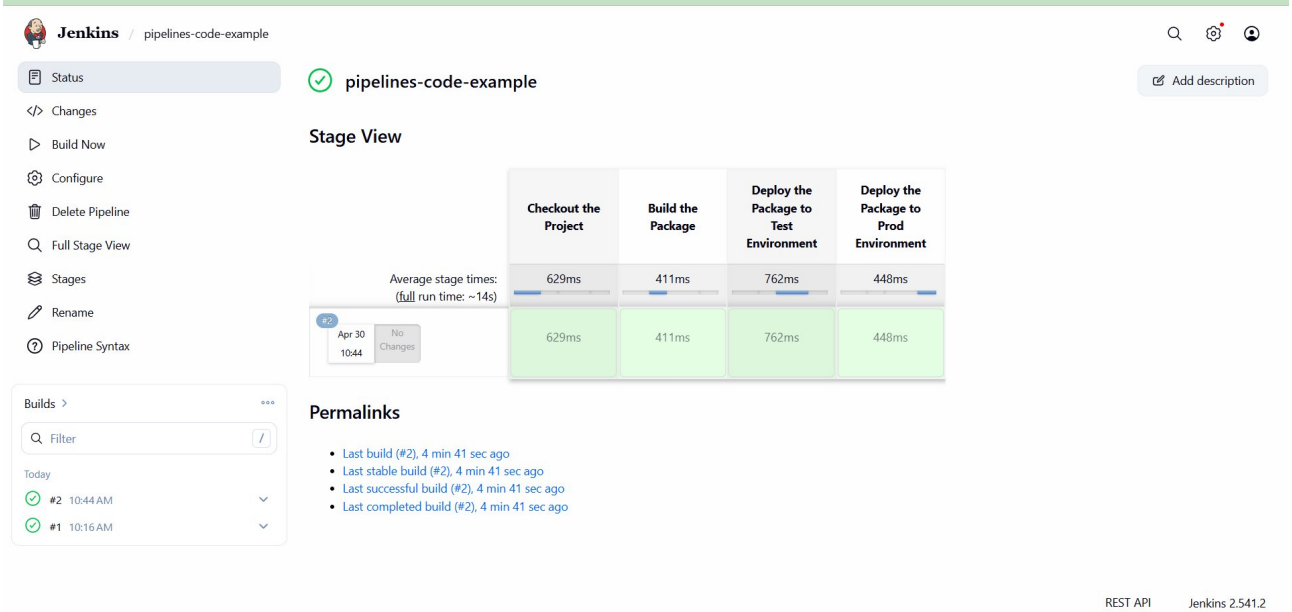
```
pipelines-code-example → Build #3 → Console Output
```

The full execution trace for Build **#3**:

```
Started by user Saleem Ahmed
[Pipeline] Start of Pipeline
[Pipeline] node
Running on Jenkins in /var/lib/jenkins/workspace/pipelines-code-example
[Pipeline] {
[Pipeline] stage
[Pipeline] { (Checkout the Project)
[Pipeline] sh
+ date
Thu Apr 30 10:57:24 AM WAT 2026
[Pipeline] sh
+ pwd
/var/lib/jenkins/workspace/pipelines-code-example
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Build the Package)
[Pipeline] echo
Build the Package
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Deploy the Package to Test Environment)
[Pipeline] echo
Deploy the Package to Test Environment
[Pipeline] }
```



■ *Console Output for pipelines-code-example Build #3 — full pipeline trace: Start of Pipeline, node allocation, workspace path confirmation, sh date outputting Thu Apr 30 10:57:24 AM WAT 2026, sh pwd outputting workspace path, each subsequent stage executing in sequence*



■ *Same Console Output in wider browser — complete execution log clearly readable, Pipeline step annotations in blue, command output in black, Download / Copy / View as plain text controls visible in header*

Decoding the Console Output Annotations

Annotation	Meaning
`[Pipeline] Start of Pipeline`	Jenkins begins parsing and executing the Groovy script

`[Pipeline] node`	Jenkins allocates an executor on an available agent node
`Running on Jenkins in /var/lib/...`	Workspace path where the build runs — derived from job name
`[Pipeline] stage`	Jenkins enters a stage() block
`[Pipeline] { (Stage Name) }`	Opening brace of a named stage — name matches stage('...')
`[Pipeline] sh`	A shell step (sh) is about to be executed
`+ date`	The + prefix (bash xtrace) marks the shell command being run
`Thu Apr 30 10:57:24 AM WAT 2026`	Actual stdout output of the date command — no + prefix
`[Pipeline] echo`	A Jenkins echo step is executing — no shell involved
`Build the Package`	Text printed by echo — no + prefix because it is not a shell command
`[Pipeline] // stage`	Closing of a stage block — the // mirrors the opening {

7. Key Lessons Learned

The following principles summarise the core insights from building and running a Declarative Pipeline in Jenkins.

1. Pipeline as Code Makes CI/CD Version-Controllable

A Declarative Pipeline in a Jenkinsfile can be stored in the same Git repository as the application. Every pipeline change is a commit — with history, diffs, code review, and rollback. This is fundamentally more maintainable than clicking through a Freestyle job UI that leaves no audit trail.

2. The Four Required Blocks Are Non-Negotiable

Every valid Declarative Pipeline must have: pipeline { }, agent, stages { }, and at least one stage() with a steps { } block. Missing any one of these causes Jenkins to reject the script at parse time before a single command is executed.

3. Stage Names Appear Directly in the Stage View

Whatever string you pass to stage('...') becomes the column header in the Stage View UI. Name stages after what they actually do — Checkout the Project, Build the Package — not generic labels like Stage 1. Good stage names make the Stage View a self-documenting delivery dashboard.

4. sh Executes Shell Commands — echo Does Not

sh 'command' spawns a shell process on the Jenkins agent and runs the command. echo 'text' is a Jenkins built-in that prints to the log without touching the shell. Use sh for anything that needs the OS: Maven builds, file copies, curl calls, deployments. Use echo for simple log annotations.

5. The + Prefix in Console Output Marks Shell Commands

When Jenkins runs sh 'date', the Console Output shows '+ date' before the command's output. This + is bash's xtrace notation — it marks the command that was executed. Lines without + are either Jenkins framework annotations or stdout from the command itself. Understanding this makes log reading much faster.

6. The Stage View Provides Instant Build Health at a Glance

Green cells = passed. Red cells = failed. The column headers show per-stage timing, and the average row tracks trends across runs. A single look at the Stage View tells you which stage broke a build and how long each phase took — without opening a single log file.

7. agent any Means Any Available Executor

agent any runs the pipeline on whichever executor is free. In a multi-node Jenkins setup, this could be a dedicated build agent, a Docker container, or a cloud-provisioned VM. For production pipelines, use agent { label 'my-label' } to pin specific stages to specific node types.

8. Pipelines Are Restartable from Any Stage

The Restart from Stage option in the build's left sidebar lets you re-run a pipeline starting from any stage that previously passed — without re-running expensive earlier stages. This is invaluable when a later stage fails due to a transient issue and the earlier steps were time-consuming.

8. Quick Reference — Declarative Pipeline Syntax

A concise reference of the most commonly used Declarative Pipeline directives and steps covered in this lab.

Syntax	Type	Description
`pipeline { }`	Block	Required outer wrapper
`agent any`	Directive	Run on any available executor
`agent { label 'x' }`	Directive	Run only on agents with label 'x'
`stages { }`	Block	Required wrapper for all stage() blocks
`stage('Name') { }`	Block	Named stage — appears in Stage View
`steps { }`	Block	Container for commands inside a stage
`echo 'text'`	Step	Print text to Console Output (no shell)
`sh 'command'`	Step	Execute a shell command on the agent
`checkout scm`	Step	Clone the SCM repo configured for the job
`environment { VAR='val' }`	Directive	Define environment variables for all stages
`post { always { } }`	Block	Steps to run after the pipeline completes

Jenkins Mastery · Saleem Ahmed · linkedin.com/in/saleemfai · Cameroon